

ECSE 420 Assignment 2

March 27, 2026

Alisha Malik 261071602

Vivek Motta 260962299

Question 1

Question 1.1

The class `FilterLock` implements the Filter algorithm from Chapter 2: Mutual Exclusion. It uses two `AtomicIntegerArray` fields: `level`, where `level[i]` stores the current waiting level of thread `i`, and `victim`, where `victim[L]` stores the last thread to declare itself the victim at level `L`. The constructor initializes both arrays to zero for all threads.

The `lock(int me)` method steps through levels 1 to `n-1` in a loop. At each level `L`, the thread sets `level[me] = L` to announce its intention to enter that level, then sets `victim[L] = me` to yield priority to other threads. It then busy-waits in a spin loop until there is no other thread `k` with `level[k] >= L` while `victim[L] == me`. The thread advances to the next level only when that condition clears. The `unlock(int me)` method simply resets `level[me]` to 0, which makes the thread invisible to all spinning threads at every level.

Question 1.2

To verify mutual exclusion, `testMutualExclusion(int n, boolean useFilter)` creates `n` threads that each perform `MUTEX_ITERATIONS = 10,000` increments of a shared integer counter under lock protection. The critical section consists of a single read-modify-write of `counter[0]++`. If any two threads are ever in the critical section simultaneously, some increments will be lost and the final counter value will be less than $n \times \text{MUTEX_ITERATIONS}$. The test returns true only if the final value equals exactly $n \times \text{MUTEX_ITERATIONS}$.

Each thread receives a unique ID in `[0, n-1]` via a `ThreadLocal<Integer>` backed by an `AtomicInteger` counter that is reset before each test. This ensures that every thread passes the correct `me` argument to `lock` and `unlock` without relying on `Thread.getId()`, which is not guaranteed to be contiguous. The test was run for every value of `n` from 2 to 8 and passed in all cases, confirming that the Filter lock provides `n`-thread mutual exclusion across the full range.

To run code:

```
javac -d bin ca/mcgill/ecse420/a2/*.java
java -cp bin ca.mcgill.ecse420.a2.FilterLockTest
```

Sample Code output:

1.2 Filter Lock: Mutual Exclusion Test (n = 2..8)

```
n = 2 : PASS
n = 3 : PASS
n = 4 : PASS
n = 5 : PASS
n = 6 : PASS
n = 7 : PASS
n = 8 : PASS
```

Question 1.3

In theory, the Filter lock does not guarantee bounded waiting, meaning it allows threads to be overtaken an arbitrary number of times. At each level, the victim mechanism ensures that at least one waiting thread makes progress, but it provides no guarantee about which thread advances. A thread that arrives at a level and sets itself as victim can be immediately overwritten as victim by a later-arriving thread, allowing that later thread to proceed first. Because the Filter lock has no doorway section that captures a thread's arrival order before the spinning begins, there is no bound on how many times a thread can be bypassed by later arrivals. The lock guarantees starvation freedom, meaning every thread eventually enters the critical section, but it permits arbitrarily unfair orderings along the way.

Question 1.4

To observe overtaking empirically, `testOvertaking(int n, boolean useFilter)` tracks two global sequence numbers: `requestSeq`, which is incremented atomically just before a thread calls `lock`, and `acquireSeq`, which is incremented atomically just after `lock` returns. Each lock acquisition produces a record of the form (`requestNumber`, `acquireNumber`, `threadId`). After all threads complete, an overtake is counted whenever thread *i* has a lower request number than thread *j* but a higher acquire number, meaning thread *i* joined the queue first but thread *j* entered the critical section first.

With $n = 8$ threads and `OVERTAKE_ITERATIONS = 500` lock attempts per thread, the test detected overtaking in the Filter lock. At least one thread that requested the lock earlier was bypassed by a later-arriving thread, confirming the theoretical prediction that the Filter lock does not enforce any arrival-order guarantee. This is consistent with the absence of a doorway section: two threads can set and overwrite `victim[L]` at the same level in a way that consistently disadvantages one of them across multiple rounds.

To run code:

```
javac -d bin ca/mcgill/ecse420/a2/*.java
java -cp bin ca.mcgill.ecse420.a2.FilterLockTest
```

Sample Code output:

1.2 Filter Lock: Mutual Exclusion Test ($n = 2..8$)

```
n = 2 : PASS
n = 3 : PASS
n = 4 : PASS
n = 5 : PASS
n = 6 : PASS
n = 7 : PASS
n = 8 : PASS
```

1.4 Filter Lock: Overtaking Observation ($n = 8$)

Total lock attempts : 4000

Overtakes observed : 878

Overtaking detected: threads were bypassed by later-arriving threads.

This confirms the Filter lock does not guarantee FCFS ordering.

Question 1.5

The class BakeryLock implements Lamport's Bakery algorithm from Chapter 2: Mutual Exclusion. It uses two AtomicIntegerArray fields: flag, where $\text{flag}[i] = 1$ indicates that thread i is currently interested in acquiring the lock, and label, where $\text{label}[i]$ holds the ticket number taken by thread i . The constructor initializes all flags to 0 and all labels to 0.

The `lock(int me)` method begins with a doorway section that sets $\text{flag}[\text{me}] = 1$ and then reads all current label values to compute a new ticket as $\max(\text{label}[0], \dots, \text{label}[n-1]) + 1$, which is stored in $\text{label}[\text{me}]$. The waiting section then iterates over every other thread k and spins until either $\text{flag}[k] == 0$ or the pair $(\text{label}[k], k)$ is lexicographically greater than $(\text{label}[\text{me}], \text{me})$, meaning thread me has strictly higher priority than thread k . Ties in label value are broken by thread ID, ensuring a total order. The `unlock(int me)` method sets $\text{flag}[\text{me}] = 0$, which immediately removes the thread from consideration by all other spinning threads.

Question 1.6

Mutual exclusion for the Bakery lock is verified using the same `testMutualExclusion` method described in Question 1.2, with `useFilter = false`. The same $n \times \text{MUTEX_ITERATIONS}$ counter check applies. The test was run for every value of n from 2 to 8 and passed in all cases, confirming that the Bakery lock provides correct n -thread mutual exclusion across the full range.

To run code:

```
javac -d bin ca/mcgill/ecse420/a2/*.java
java -cp bin ca.mcgill.ecse420.a2.BakeryLockTest
```

1.6 Bakery Lock: Mutual Exclusion Test ($n = 2..8$)

$n = 2$: PASS
 $n = 3$: PASS
 $n = 4$: PASS
 $n = 5$: PASS
 $n = 6$: PASS
 $n = 7$: PASS
 $n = 8$: PASS

Question 1.7

In theory, the Bakery lock guarantees first-come-first-served ordering and therefore does not allow arbitrary overtaking. The key property is that the doorway section is finite: it completes in a bounded number of steps regardless of what other threads are doing. If thread A completes its doorway before thread B begins its doorway, then A's label is assigned before B reads any label values, which means B will observe A's label and assign itself a strictly larger value. Because the waiting condition is based on lexicographic comparison of (label, threadId) pairs, B will always spin until A has entered and exited the critical section. This is the formal FCFS property: once A's doorway precedes B's doorway, A's critical section also precedes B's critical section. Unlike the Filter lock, overtaking is theoretically impossible in the Bakery lock.

Question 1.8

The same testOvertaking method was applied to the Bakery lock with $n = 8$ threads and `OVERTAKE_ITERATIONS = 500` lock attempts per thread. The result showed zero overtakes detected across all 4,000 total lock acquisitions. No thread that requested the lock earlier was bypassed by a later-arriving thread, which is consistent with the FCFS guarantee established in Question 1.7. The empirical observation directly matches the theoretical prediction: the doorway section assigns ticket numbers that enforce a strict arrival order, making it impossible for a later thread to enter the critical section before an earlier one in any execution that the test could observe.

To run code:

```
javac -d bin ca/mcgill/ecse420/a2/*.java
java -cp bin ca.mcgill.ecse420.a2.BakeryLockTest
```

1.6 Bakery Lock: Mutual Exclusion Test ($n = 2..8$)

n = 2 : PASS
n = 3 : PASS
n = 4 : PASS
n = 5 : PASS
n = 6 : PASS
n = 7 : PASS
n = 8 : PASS

1.8 Bakery Lock: Overtaking Observation ($n = 8$)

Total lock attempts : 4000
Overtakes observed : 0
No overtaking detected. Consistent with FCFS guarantee.

Question 2

2.1 Does the Shaky protocol satisfy mutual exclusion?

Answer: Yes, the protocol does satisfy mutual exclusion.

Proof by contradiction: Suppose both A and B are in the CS at the same time. To get there, each thread has to (i) see `busy == false` in the inner loop, (ii) set `busy = true`, and (iii) see `turn == me` when checking the outer while.

Say A wrote $turn = A$ after B's last write of $turn = B$. That means A saw $busy == false$, set $busy = true$, then checked $turn$ and saw A, so A enters the CS.

Now for B to also be in the CS, B needs to see $turn == B$ eventually. But think about what B is doing at this point, B is already set $busy = true$ at some earlier point and saw $turn != B$, which sent it back to the outer loop. When B tries to spin in the inner loop again, it's checking $busy$, which is now $true$ (A set it). So B is stuck spinning there and never gets to check $turn$ again.

B can't see $turn == B$ while it's trapped in the inner loop, which is a contradiction. So they can't both be in the CS. ■

2.2 . Is this protocol deadlock-free?

No this protocol is not deadlock-free, here is an execution that produces a deadlock:

Step	Thread A	Thread B	turn	busy
1	writes $turn = A$	—	A	False
2	sees $busy == false$, breaks out of inner loop	—	A	False
3	—	Writes $turn = B$	B	False
4	sets $busy = true$	—	B	True
5	checks $turn$, sees B not A, goes back to outer loop	—	B	True
6	back in inner loop, sets $turn = A$, but $busy == true$ so it just keeps spinning	Gets to inner loop, sees $busy == true$, keeps spinning	Keeps changing	True

At step 6 both threads are stuck spinning in the inner loop. They keep writing to $turn$ but $busy$ never goes back to false because neither thread is actually in the CS, meaning $unlock()$ never gets called. Neither thread can move forward. That's a deadlock.

2.3. Is this protocol starvation-free?

Since we already showed the protocol can deadlock, starvation-freedom is automatically out the window too, you can't have starvation-freedom without deadlock-freedom. In the execution above both threads are stuck spinning forever and neither one ever gets into the CS, so they both starve.

Question 3

History A

Linearizability:

No. The reason is that B's read(2) starts after C's write(3) has already finished, which you can see from the timeline since B's read is the rightmost operation. This means in any valid linearization, C.write(3) must be placed before B.read(2). But then the last write B sees should be 3, not 2. Since B reads 2, no valid linearization exists.

Sequential consistency:

Yes. We don't need real-time ordering here, just program order within each thread. The ordering below is legal:

$A.w(0) \rightarrow B.w(1) \rightarrow A.r(1) \rightarrow A.w(2) \rightarrow C.r(2) \rightarrow B.r(2) \rightarrow C.w(3)$

Checking each read against the last write before it:

- A.r(1) sees B.w(1) → correct
- C.r(2) sees A.w(2) → correct
- B.r(2) sees A.w(2) → correct

Program order within A, B, C is all preserved. History A is sequentially consistent.

History B

Sequential consistency:

No. Look at what the reads require:

- C.r(1) returning 1 forces B.w(1) to come after C.w(2) in any valid ordering. If C.w(2) came after B.w(1), C would read 2, not 1
- B.r(2) returning 2 forces C.w(2) to come after B.w(1). Otherwise B would read 1

So we need C.w(2) before B.w(1) AND B.w(1) before C.w(2) at the same time. That's a cycle and no ordering can satisfy both.

Linearizability:

No. Linearizability is stricter than sequential consistency. Since History B doesn't even pass sequential consistency, it can't be linearizable.

Question 4

4.1 - Is division by zero possible?

No it is not. The volatile keyword on *v* sets up a happens-before chain that prevents it:

- Within writer(): $x = 42$ executes before $v = \text{true}$ (program order)
- The JMM says a volatile write happens-before any thread that subsequently reads that same volatile variable
- Combining these: by the time reader sees $v == \text{true}$, it is guaranteed to already see $x == 42$

The JMM also stops the compiler/JVM from floating the write to *v* above the write to *x*. So *x* will always be 42 by the time the division runs. No divide by zero.

4.2

Both volatile: Both volatile: Still no divide by zero. The happens-before chain from 4.1 still holds because a volatile write to any variable still happens-before a subsequent volatile read of that same variable, and program order within `writer()` still guarantees `x = 42` runs before `v = true`. Making `x` volatile too only adds stronger guarantees on top.

Neither volatile: Divide by zero becomes possible through two different paths:

- The compiler or JVM is free to reorder writer's instructions, so `v = true` could become visible to other threads before `x = 42` ever gets written
- Even without reordering, CPU caches mean the reader could be looking at a stale `v = true` and a stale `x = 0` from its own cache line. The write to `x = 42` might still be sitting in the writer's cache and not flushed to main memory yet.

Either way, the reader can enter the if-block and hit `100/0`.

Question 5

LockOne: Thread *i* sets `flag[i] = true`, then waits while `flag[j] == true`. The safety argument relies on: if I see `flag[j] == false`, *j* hasn't flagged itself yet, so I'm safe to go in.

With regular registers, though, a read that's concurrent with a write is allowed to return the stale old value. So consider this execution:

Step	Thread A	Thread B
1	writing <code>flag[A] = true</code> , still in progress	writing <code>flag[B] = true</code> , still in progress
2	reads <code>flag[B]</code> during B's write, gets stale false	reads <code>flag[A]</code> during A's write, gets stale false
3	sees false, enters CS	sees false, enters CS

Both threads end up in the CS at the same time, so mutual exclusion is violated. Mutual exclusion **fails** with regular registers.

LockTwo: Thread *i* writes `victim = i` then spins while `victim == i`. There's only one shared register and two threads writing to it.

Say both A and B are in the CS at the same time, meaning both saw `victim != me` at some point. Only two writes to `victim` ever happen total, so one of them had to finish after the other. Say B's write was last. B reads `victim` after its own write, which also means after A's write since B's was the later one. At that point no write is ongoing, and a regular register with no concurrent write has to return the most recently written value, which is `victim = B`. So B sees its own value and has to keep spinning. But we assumed B was in the CS, which is a contradiction. ■

So whichever thread wrote last will always read its own value back and get stuck waiting. This works even with regular registers because by the time the last writer reads, there's nothing else writing concurrently. Mutual exclusion **holds** with regular registers.

Question 6

Question 6.1

It is **False** because the modified construction is not a regular M-valued MRSW register. In fact, it fails to satisfy even the weaker safe register property, which requires that a non-overlapping read always returns the last written value.

To understand why, it is necessary to examine the invariant that the original construction maintains. In the original code, `write(x)` sets `r_bit[x]` to true and then clears all bits below `x` (indices `x-1` down to 0). This ensures that after any complete write, the lowest true bit in the array is exactly the last written value, which is what `read()` returns by scanning upward from index 0. The modified loop changes this behavior so that `write(x)` instead clears all bits above `x`, leaving all lower-indexed bits untouched.

This breaks the construction immediately. Consider the following sequential execution with no concurrency at all. The constructor initializes `r_bit[0]` to true and all other bits to false, representing the initial value 0. A call to `write(2)` sets `r_bit[2]` to true and then clears `r_bit[3]`, `r_bit[4]`, and so on. However, `r_bit[0]` is never cleared. A subsequent non-overlapping `read()` scans from index 0, finds `r_bit[0]` still true, and returns 0 instead of 2. Since this non-overlapping read returns a value that was not the last written value, the safe register condition is violated, and therefore the regular register condition is also violated.

Question 6.2

It is **False** because the modified construction does not yield a safe M-valued MRSW register. As demonstrated in the analysis for question 6.1, the construction fails the safety property in a purely sequential execution with no concurrency. Safety requires that any read that does not overlap a write must return the most recently written value. The counterexample in 6.1, where `write(2)` is followed by a non-overlapping `read()` that returns 0, is a direct violation of this requirement. Since the failure occurs even without any interleaving of concurrent operations, no argument about concurrent behavior can rescue the construction. The modified loop leaves lower-indexed bits from prior writes or initialization permanently set, so the lowest true bit no longer reflects the last written value, and `read()` consistently returns incorrect results in sequential use.

Question 7

We prove by reduction that if binary consensus is impossible for two threads using only atomic read/write registers, then it is also impossible for any $n > 2$ threads.

Suppose for contradiction that there exists a protocol `P` that solves binary consensus for $n > 2$ threads using only atomic registers. We construct a two-thread protocol `P'` as follows. Thread A in `P'` simulates the behavior of threads 0 through $n-2$ from `P`, executing their steps sequentially in a single-threaded loop. Thread B in `P'` simulates thread $n-1$ from `P` exactly as written. Both threads in `P'` operate on the same shared atomic registers used by `P`.

Every execution of `P'` corresponds to a valid interleaved execution of `P`, because all reads and writes performed by A and B are legal atomic register operations. Since `P` is assumed to solve consensus, every such execution reaches agreement on

a value that equals one of the proposed inputs. Therefore P' also solves binary consensus. But this contradicts the given result that binary consensus for two threads is impossible using atomic registers. The contradiction implies that no such P can exist for any $n > 2$. ■

Question 8

We prove by reduction that if binary consensus is impossible for n threads using only atomic read/write registers, then consensus over k values is also impossible for the same n threads, for any $k > 2$.

Suppose for contradiction that there exists a protocol Q that solves k -valued consensus for n threads using only atomic registers, where $k > 2$. We construct a binary consensus protocol Q' for n threads as follows. Each thread receives a binary input $b \in \{0, 1\}$. Since $k > 2$, both 0 and 1 are valid values in the k -valued domain, so each thread simply proposes its binary input directly to Q without any transformation.

Since Q is a valid consensus protocol, it guarantees agreement, meaning all threads decide the same value v , and validity, meaning v must equal one of the proposed inputs. Because all proposed inputs are drawn from $\{0, 1\}$, the decided value v must also be in $\{0, 1\}$. Therefore Q' correctly solves binary consensus for n threads using only atomic registers. But this contradicts the result from Question 7, which establishes that binary consensus for n threads using atomic registers is impossible. The contradiction implies that no such Q can exist for any $k > 2$. ■

Appendix: Source Code

Question 1 Code:

BakeryLock

```
1 package ca.mcgill.ecse420.a2;
2
3 import java.util.concurrent.atomic.AtomicIntegerArray;
4
5 //Lamport's bakery lock for n-thread mutual exclusion
6 public class BakeryLock {
7
8     private final int n;
9     private final AtomicIntegerArray flag; //flag[i] = 1 if thread i wants the lock
10    private final AtomicIntegerArray label; //label[i] = ticket number for thread i
11
12    //initialize all flags and labels to 0
13    public BakeryLock(int n) {
14        this.n = n;
15        flag = new AtomicIntegerArray(n);
16        label = new AtomicIntegerArray(n);
17    }
18
19    //acquire lock: doorway takes a ticket, then spins until all lower-priority threads have passed
20    public void lock(int me) {
21        //doorway: announce interest and take a ticket
22        flag.set(me, 1);
23        int maxLabel = 0;
24        for (int k = 0; k < n; k++) {
25            int lk = label.get(k);
26            if (lk > maxLabel) maxLabel = lk;
27        }
28        label.set(me, maxLabel + 1);
29
30        //wait until every other interested thread has lower priority (higher label or higher id)
31        for (int k = 0; k < n; k++) {
32            if (k == me) continue;
33
34            //wait for thread k to finish its own doorway
35            while (flag.get(k) == 1 && label.get(k) == 0) Thread.yield();
36
37            //wait while thread k has strictly higher priority
38            while (flag.get(k) == 1 && isLexLess(label.get(k), k, label.get(me), me)) Thread.yield();
39        }
40    }
```

```
41
42    //release lock: clear flag so other threads stop waiting on this thread
43    public void unlock(int me) {
44        flag.set(me, 0);
45    }
46
47    //returns true if (labelA, a) < (labelB, b) lexicographically, meaning A has higher priority
48    private boolean isLexLess(int labelA, int a, int labelB, int b) {
49        return labelA < labelB || (labelA == labelB && a < b);
50    }
51 }
52
```

FilterLock

```

1 package ca.mcgill.ecse420.a2;
2
3 import java.util.concurrent.atomic.AtomicIntegerArray;
4
5 //filter lock for n-thread mutual exclusion
6 public class FilterLock {
7
8     private final int n;
9     private final AtomicIntegerArray level; //level[i] = current level of thread i
10    private final AtomicIntegerArray victim; //victim[L] = last thread to set itself victim at level L
11
12    //initialize both arrays to zero for all n threads
13    public FilterLock(int n) {
14        this.n = n;
15        level = new AtomicIntegerArray(n);
16        victim = new AtomicIntegerArray(n);
17    }
18
19    //acquire lock: thread advances through levels 1..n-1, spinning at each until no longer victim
20    public void lock(int me) {
21        for (int L = 1; L < n; L++) {
22            level.set(me, L);
23            victim.set(L, me);
24
25            //spin while another thread is at level >= L and this thread is still the victim
26            boolean waiting;
27            do {
28                waiting = false;
29                for (int k = 0; k < n; k++) {
30                    if (k != me && level.get(k) >= L && victim.get(L) == me) {
31                        waiting = true;
32                        break;
33                    }
34                }
35            } while (waiting);
36        }
37    }
38

```

```

39    //release lock: reset level to 0 so this thread is invisible to all spinners
40    public void unlock(int me) {
41        level.set(me, 0);
42    }
43 }
44

```

BakeryLockTest

```

1 package ca.mcgill.ecse420.a2;
2
3 import java.util.ArrayList;
4 import java.util.Collections;
5 import java.util.List;
6 import java.util.concurrent.atomic.AtomicInteger;
7
8 //tests for the Bakery lock: mutual exclusion and overtaking observation
9 public class BakeryLockTest {
10
11     private static final int MUTEX_ITERATIONS = 10_000; //increments per thread in mutex test
12     private static final int OVERTAKE_ITERATIONS = 500; //lock attempts per thread in overtake test
13     private static final int OVERTAKE_THREADS = 8; //thread count for overtake test
14
15     //assigns unique IDs 0..n-1 to threads; must be reset before each test
16     private static final AtomicInteger idCounter = new AtomicInteger(0);
17
18     //each thread gets its own unique ID via ThreadLocal
19     private static final ThreadLocal<Integer> threadId =
20         ThreadLocal.withInitial(() -> idCounter.getAndIncrement());
21
22     public static void main(String[] args) throws InterruptedException {
23
24         //1.6: mutual exclusion test for n = 2..8
25         System.out.println("1.6 Bakery Lock: Mutual Exclusion Test (n = 2..8)");
26         for (int n = 2; n <= 8; n++) {
27             boolean passed = testMutualExclusion(n);
28             System.out.printf(" n = %d : %s\n", n, passed ? "PASS" : "FAIL");
29         }
30
31         //1.8: overtaking observation with n = 8
32         System.out.println("\n1.8 Bakery Lock: Overtaking Observation (n = 8)");
33         testOvertaking(OVERTAKE_THREADS);
34     }
35
36     //1.6: mutual exclusion test: n threads each increment a shared counter MUTEX_ITERATIONS times.
37     //if no mutual exclusion violation occurs, final value == n * MUTEX_ITERATIONS.
38     private static boolean testMutualExclusion(int n) throws InterruptedException {
39         idCounter.set(0);
40         final int[] counter = {0}; //shared counter: lost updates indicate a mutex violation
41         BakeryLock lock = new BakeryLock(n);

```

```

43         Thread[] threads = new Thread[n];
44         for (int t = 0; t < n; t++) {
45             threads[t] = new Thread(() -> {
46                 int me = threadId.get();
47                 for (int i = 0; i < MUTEX_ITERATIONS; i++) {
48                     lock.lock(me);
49                     counter[0]++; //critical section
50                     lock.unlock(me);
51                 }
52             });
53         }
54
55         for (Thread t : threads) t.start();
56         for (Thread t : threads) t.join();
57
58         return counter[0] == n * MUTEX_ITERATIONS;
59     }
60
61     //1.8: overtaking observation test: records the order threads request the lock vs the order they acquire it.
62     //an overtake = thread i requested first but thread j acquired first.
63     private static void testOvertaking(int n) throws InterruptedException {
64         idCounter.set(0);
65
66         AtomicInteger requestSeq = new AtomicInteger(0); //stamped just before lock()
67         AtomicInteger acquireSeq = new AtomicInteger(0); //stamped just after lock() returns
68         List<int[]> records = Collections.synchronizedList(new ArrayList<>()); // (request, acquire, id)
69
70         BakeryLock lock = new BakeryLock(n);
71
72         Thread[] threads = new Thread[n];
73         for (int t = 0; t < n; t++) {
74             threads[t] = new Thread(() -> {
75                 int me = threadId.get();
76                 for (int i = 0; i < OVERTAKE_ITERATIONS; i++) {
77                     int myRequest = requestSeq.getAndIncrement(); //record request order
78                     lock.lock(me);
79                     int myAcquire = acquireSeq.getAndIncrement(); //record acquire order
80                     records.add(new int[]{myRequest, myAcquire, me});
81                     lock.unlock(me);

```

```

82     }
83     });
84 }
85
86 for (Thread t : threads) t.start();
87 for (Thread t : threads) t.join();
88
89 //count pairs where i requested before j but j acquired before i
90 int overtakeCount = 0;
91 int total = records.size();
92 for (int i = 0; i < total; i++) {
93     for (int j = 0; j < total; j++) {
94         if (i == j) continue;
95         int[] ri = records.get(i), rj = records.get(j);
96         if (ri[0] < rj[0] && ri[1] > rj[1]) overtakeCount++;
97     }
98 }
99
100 System.out.printf(" Total lock attempts : %d\n", total);
101 System.out.printf(" Overtakes observed : %d\n", overtakeCount);
102 if (overtakeCount > 0) {
103     System.out.println(" Overtaking DETECTED: unexpected, Bakery should be FCFS.");
104 } else {
105     System.out.println(" No overtaking detected. Consistent with FCFS guarantee.");
106 }
107 }
108 }
109

```

FilterLockTest

```

1 package ca.mcgill.ecse420.a2;
2
3 import java.util.ArrayList;
4 import java.util.Collections;
5 import java.util.List;
6 import java.util.concurrent.atomic.AtomicInteger;
7
8 //tests for the Filter lock: mutual exclusion and overtaking observation
9 public class FilterLockTest {
10
11     private static final int MUTEX_ITERATIONS = 10_000; //increments per thread in mutex test
12     private static final int OVERTAKE_ITERATIONS = 500; //lock attempts per thread in overtake test
13     private static final int OVERTAKE_THREADS = 8; //thread count for overtake test (1.4)
14
15     //assigns unique IDs 0..n-1 to threads; must be reset before each test
16     private static final AtomicInteger idCounter = new AtomicInteger(0);
17
18     //each thread gets its own unique ID via ThreadLocal
19     private static final ThreadLocal<Integer> threadId =
20         ThreadLocal.withInitial(() -> idCounter.getAndIncrement());
21
22     public static void main(String[] args) throws InterruptedException {
23
24         //1.2: Mutual exclusion test for n = 2..8
25         System.out.println("1.2 Filter Lock: Mutual Exclusion Test (n = 2..8)");
26         for (int n = 2; n <= 8; n++) {
27             boolean passed = testMutualExclusion(n);
28             System.out.printf(" n = %d : %s\n", n, passed ? "PASS" : "FAIL");
29         }
30
31         //1.4: Overtaking observation with n = 8
32         System.out.println("\n1.4 Filter Lock: Overtaking Observation (n = 8)");
33         testOvertaking(OVERTAKE_THREADS);
34     }
35
36     //1.2: Mutual exclusion test: n threads each increment a shared counter MUTEX_ITERATIONS times.
37     //if no mutual exclusion violation occurs, final value == n * MUTEX_ITERATIONS.
38     private static boolean testMutualExclusion(int n) throws InterruptedException {
39         idCounter.set(0);
40         final int[] counter = {0}; //shared counter: lost updates indicate a mutex violation
41         FilterLock lock = new FilterLock(n);

```

```

43 Thread[] threads = new Thread[n];
44 for (int t = 0; t < n; t++) {
45     threads[t] = new Thread() -> {
46         int me = threadId.get();
47         for (int i = 0; i < MUTEX_ITERATIONS; i++) {
48             lock.lock(me);
49             counter[0]++; //critical section
50             lock.unlock(me);
51         }
52     };
53 }
54
55 for (Thread t : threads) t.start();
56 for (Thread t : threads) t.join();
57
58 return counter[0] == n * MUTEX_ITERATIONS;
59 }
60
61 //1.4: Overtaking observation test - records the order threads request the lock vs the order they acquire it.
62 // An overtake = thread i requested first but thread j acquired first.
63 private static void testOvertaking(int n) throws InterruptedException {
64     idCounter.set(0);
65
66     AtomicInteger requestSeq = new AtomicInteger(0); //stamped just BEFORE lock()
67     AtomicInteger acquireSeq = new AtomicInteger(0); //stamped just AFTER lock() returns
68     List<int[]> records = Collections.synchronizedList(new ArrayList<>()); // (request, acquire, id)
69
70     FilterLock lock = new FilterLock(n);
71
72     Thread[] threads = new Thread[n];
73     for (int t = 0; t < n; t++) {
74         threads[t] = new Thread() -> {
75             int me = threadId.get();
76             for (int i = 0; i < OVERTAKE_ITERATIONS; i++) {
77                 int myRequest = requestSeq.getAndIncrement(); //record request order
78                 lock.lock(me);
79                 int myAcquire = acquireSeq.getAndIncrement(); //record acquire order
80                 records.add(new int[]{myRequest, myAcquire, me});

```

```

79                 int myAcquire = acquireSeq.getAndIncrement(); //record acquire order
80                 records.add(new int[]{myRequest, myAcquire, me});
81                 lock.unlock(me);
82             }
83         };
84     }
85
86     for (Thread t : threads) t.start();
87     for (Thread t : threads) t.join();
88
89     //count pairs where i requested before j but j acquired before i
90     int overtakeCount = 0;
91     int total = records.size();
92     for (int i = 0; i < total; i++) {
93         for (int j = 0; j < total; j++) {
94             if (i == j) continue;
95             int[] ri = records.get(i), rj = records.get(j);
96             if (ri[0] < rj[0] && ri[1] > rj[1]) overtakeCount++;
97         }
98     }
99
100     System.out.printf(" Total lock attempts : %d\n", total);
101     System.out.printf(" Overtakes observed : %d\n", overtakeCount);
102     if (overtakeCount > 0) {
103         System.out.println(" Overtaking detected: threads were bypassed by later-arriving threads.");
104         System.out.println(" This confirms the Filter lock does not guarantee FCFS ordering.");
105     } else {
106         System.out.println(" No overtaking detected in this run.");
107     }
108 }
109 }
110

```